

Memoria de Trabajo de Fin de Grado

Lockstrm

*Plataforma B2B privada de streaming de vídeo con gestión
de acceso por roles y analíticas de visualización*

Autor: Manuel Anniji

Grado: Ingeniería Informática

Curso académico: 2025 / 2026

1. Título del proyecto

A la hora de elegir un título académico para un proyecto se busca un equilibrio entre precisión técnica, capacidad de comunicar el valor del trabajo y formalidad. A continuación se proponen tres alternativas, cada una con un matiz distinto:

- **Opción 1 — orientada al producto:** «Lockstrm: diseño e implementación de una plataforma B2B de streaming privado de vídeo con control de acceso basado en roles y analíticas de visualización».
- **Opción 2 — orientada a la arquitectura:** «Arquitectura segura para distribución privada de contenido audiovisual: aplicación full-stack con Spring Boot, Angular y autenticación JWT sobre streaming HTTP por rangos».
- **Opción 3 — orientada a la analítica y el control:** «Lockstrm: sistema integral de gobierno de contenido audiovisual corporativo, con métricas de visualización y gestión granular de permisos por grupo».

El título finalmente escogido debería reflejar tanto el carácter B2B y privado de la plataforma como el aporte técnico diferencial: la combinación de seguridad, granularidad de roles y observabilidad del consumo.

3. Estudio del sector productivo

El mercado del vídeo bajo demanda ha pasado, en poco más de una década, de ser un nicho experimental a convertirse en uno de los principales canales de consumo digital. Aunque el imaginario colectivo tiende a asociar el streaming con grandes plataformas B2C como Netflix, Amazon Prime Video o Disney+, en paralelo ha emergido un segmento mucho menos visible para el gran público pero estratégicamente crítico para muchas organizaciones: el streaming privado B2B, también conocido como *private video hosting* o *enterprise video platform* (EVP).

Empresas como Vimeo OTT, Wistia, Brightcove, Kaltura o Panopto llevan años proporcionando infraestructura para que organizaciones distribuyan contenido audiovisual entre audiencias controladas: equipos internos en programas de formación, clientes con acceso a vídeos protegidos por licencia, comunidades de afiliados, redes de franquicias o entornos educativos corporativos. La demanda ha crecido al ritmo del trabajo distribuido y de la necesidad de comunicación asíncrona, y ha consolidado un sector con dos exigencias muy claras: control estricto del acceso al contenido y trazabilidad detallada del consumo.

El primer requisito, el control de acceso, deja de ser una cuestión meramente técnica para convertirse en una cuestión legal y contractual. Las organizaciones que distribuyen formación, manuales internos, sesiones grabadas con información sensible o material sometido a derechos de autor necesitan garantizar que sólo determinados usuarios, en determinados grupos, y bajo determinados roles, pueden consumir cada pieza. Las soluciones que se limitan a ocultar el vídeo tras un enlace «no listado» no cumplen, por sí solas, los requisitos de cumplimiento que muchas empresas exigen hoy.

El segundo requisito, la analítica del consumo, se ha convertido en un elemento de valor diferencial. Saber qué usuarios han visto cada vídeo, cuánto tiempo, en qué momentos abandonan y con qué frecuencia vuelven a contenidos concretos es una información que alimenta decisiones de negocio: medir la eficacia de un plan de formación, detectar contenidos obsoletos, demostrar el cumplimiento normativo de visualizaciones obligatorias o priorizar inversión en producción audiovisual.

En este contexto, las plataformas privadas con código propietario que ofrezcan a la vez una experiencia de usuario moderna, un control fino de permisos y una capa analítica accesible siguen siendo escasas, especialmente en el mercado de pequeñas y medianas organizaciones, donde los precios por usuario de las EVP de referencia suelen ser inasumibles. Lockstrm se sitúa precisamente en ese hueco: una plataforma autoalojable, asentada en tecnologías abiertas y maduras, que permita a una organización servir vídeo de forma segura y entender cómo se consume.

4. Finalidad del proyecto

La finalidad de Lockstrm es resolver, en un único producto y con una arquitectura coherente, el problema de las organizaciones que necesitan distribuir contenido audiovisual privado entre audiencias acotadas y, al mismo tiempo, entender cómo se está consumiendo ese contenido.

Hoy una organización que quiere compartir vídeo con sus empleados, colaboradores o clientes se enfrenta normalmente a una disyuntiva incómoda. Por un lado, las plataformas públicas como YouTube o Vimeo son inmediatas y fáciles de usar, pero los mecanismos de protección que ofrecen son limitados —enlaces no listados, contraseñas a nivel de vídeo, restricciones por dominio— y, aún más importante, la información que devuelven sobre el consumo es agregada y anónima, lo cual impide auditar visualizaciones por usuario. Por otro lado, las plataformas empresariales como Brightcove, Kaltura o Vimeo OTT cubren ese vacío con creces, pero su coste por puesto y su complejidad operativa las dejan fuera del alcance de muchos equipos.

Lockstrm aborda esa brecha proporcionando una plataforma cerrada en la que cada vídeo está asociado a uno o varios grupos, cada grupo a un conjunto de miembros con roles diferenciados, y cada sesión de reproducción queda registrada con identidad del usuario, marca temporal y tiempo efectivamente visto. El resultado es un entorno donde la pregunta «¿quién ha visto qué, cuándo y cuánto?» tiene siempre una respuesta verificable, y donde la administración del acceso no exige operaciones manuales fuera de la plataforma.

La finalidad última, por tanto, no es competir frontalmente con las grandes EVP, sino demostrar que con un stack tecnológico moderno y bien escogido es posible construir un sistema que cubra el 80% de los casos de uso del streaming corporativo a una fracción del coste y con plena soberanía sobre los datos.

5. Descripción básica del proyecto

Lockstrm es una plataforma web full-stack pensada para que una organización privada pueda subir vídeos, organizarlos por grupos, decidir qué usuarios y con qué nivel de privilegio pueden acceder a cada grupo, y obtener analíticas detalladas sobre el consumo de ese contenido. Internamente se estructura como una aplicación cliente-servidor desacoplada: un backend en Spring Boot 3.2.3 sobre Java 21 que expone una API REST autenticada con JWT, una base de datos relacional MySQL como sistema de persistencia, y un cliente web en Angular 21 construido íntegramente con *Standalone Components* y *Signals*. Toda la arquitectura se entrega contenedorizada con Docker y orquestada con docker-compose, con Nginx actuando como proxy inverso, Spring Boot Actuator exponiendo métricas para Prometheus y un workflow de GitHub Actions que automatiza el despliegue por SSH a la rama main.

El usuario final accede a través de un único punto de entrada en el navegador, se autentica con credenciales y, a partir de ahí, opera dentro del ámbito de los grupos en los que es miembro. Según su rol —*SUPER_ADMIN*, *ADMIN*, *EDITOR* o *MIEMBRO*— podrá administrar la plataforma completa, gestionar un grupo, subir contenido o únicamente consumirlo. El consumo de vídeo se realiza mediante *HTTP range requests*, garantizando una experiencia de reproducción fluida con búsqueda en la línea de tiempo, y la propia reproducción emite latidos (*heartbeats*) periódicos al backend, que se persisten como telemetría por sesión y alimentan el módulo de analíticas.

6. Descripción detallada del proyecto

6.1. Herramientas de desarrollo software utilizadas

La elección del stack tecnológico en un proyecto de este tipo no es accesorio: condiciona tanto el rendimiento como la facilidad de mantenimiento y la curva de aprendizaje del equipo que herede el código. En Lockstrm se ha priorizado un conjunto de tecnologías maduras, abiertas, con comunidad activa y soporte de larga duración.

Java 21 (LTS)

Se ha optado por Java 21 como versión base por varios motivos. En primer lugar, es una versión LTS, lo que asegura soporte a largo plazo y reduce la presión por actualizar el entorno de producción cada pocos meses. En segundo lugar, incorpora mejoras significativas respecto a versiones anteriores: *pattern matching* sobre switch, *record patterns*, hilos virtuales (Project Loom) y un rendimiento del recolector de basura considerablemente mejorado. En un servicio que potencialmente atenderá muchas peticiones *range* concurrentes para streaming, contar con hilos virtuales abre la puerta a un modelo de concurrencia mucho más sencillo que el tradicional basado en pools de hilos.

Spring Boot 3.2.3

Spring Boot es, a día de hoy, la elección más natural para construir un backend Java de tamaño medio. Aporta autoconfiguración, integración nativa con Spring Data JPA, Spring Security, Actuator y Micrometer, y un ecosistema enorme de iniciadores. La versión 3.2.3 supone un punto de equilibrio adecuado: ya está consolidada sobre Jakarta EE 10 y

plenamente compatible con Java 21, sin las inestabilidades habituales de las releases más recientes. Su filosofía de «convención sobre configuración» permite enfocar el esfuerzo de desarrollo en el dominio del problema, en lugar de en cableado de infraestructura.

Spring Data JPA y Lombok

Para la capa de persistencia se ha utilizado Spring Data JPA sobre Hibernate, que permite trabajar con repositorios declarativos y proyecciones tipadas sin escribir SQL artesanal en los casos habituales. La gestión de la evolución del esquema se hace con migraciones Flyway versionadas (V1__init.sql ... V9__normalize_users.sql al cierre de esta memoria), lo que asegura que cualquier entorno parte del mismo esquema y que los cambios quedan auditables en el repositorio. Lombok complementa este enfoque eliminando el ruido del *boilerplate* (getters, setters, constructores, equals/hashCode) y permitiendo que las entidades y DTO sigan siendo expresivos y legibles.

Spring Security y JWT

Spring Security proporciona toda la infraestructura de autenticación y autorización, incluyendo la cadena de filtros HTTP, el contexto de seguridad por petición y la integración con anotaciones declarativas como `@PreAuthorize`. Sobre ella se ha construido un modelo *stateless* basado en JWT firmados con la biblioteca JWT, lo que permite escalar horizontalmente la API sin compartir sesiones.

Angular 21

En el frontend, Angular 21 representa el punto de madurez del nuevo paradigma del framework: *Standalone Components* por defecto —sin `NgModules`— y *Signals* como modelo de reactividad. La adopción de esta versión, en lugar de continuar con Angular clásico, no es meramente cosmética: simplifica el árbol de componentes, elimina capas de indirección que históricamente lastraban la legibilidad de los proyectos Angular y produce *bundles* más pequeños gracias al *tree-shaking* agresivo.

MySQL

MySQL ha sido la elección como motor relacional. En un dominio donde las relaciones entre usuarios, grupos, vídeos y permisos son centrales, una base de datos relacional encaja de forma mucho más natural que cualquier alternativa documental. MySQL aporta, además, una operativa ampliamente conocida, ecosistema de herramientas maduro y y compatibilidad total con Hibernate y Spring Data.

Docker, docker-compose y GitHub Actions

La infraestructura completa se ha contenedorizado con Docker, definiendo en `docker-compose.yml` los servicios del backend, la base de datos, el proxy inverso Nginx y, opcionalmente, Prometheus para la recolección de métricas. Esto garantiza que cualquier entorno —local, preproducción o producción— se construye de forma reproducible con un único comando. Sobre esa base, un *workflow* de GitHub Actions (`.github/workflows/deploy.yml`) automatiza el despliegue contra el servidor de producción: ante cada *push* a main establece una conexión SSH, hace git pull, reconstruye los

contenedores con docker compose up -d --build y limpia las imágenes huérfanas.

Nginx, Actuator y Prometheus

Nginx aporta terminación de TLS, redirección HTTP → HTTPS, compresión, *buffering* de respuestas y, lo que es más relevante en este proyecto, la capacidad de servir vídeo con soporte nativo para *range requests* y caché. Spring Boot Actuator expone *health checks* y métricas en formato Micrometer, que Prometheus consume periódicamente. El resultado es una plataforma con observabilidad de base, lista para integrarse con un panel Grafana o un sistema de alertas externo.

6.2. Diseño del proyecto

El modelo de datos de Lockstrm se ha diseñado siguiendo los principios clásicos del análisis entidad-relación, intentando que las decisiones de diseño físico reflejen con fidelidad la lógica del dominio. El esquema actual está consolidado en nueve migraciones Flyway (V1__init.sql a V9__normalize_users.sql) y define siete tablas con sus correspondientes restricciones de integridad e índices.

La tabla **usuarios** representa a una persona registrada en la plataforma. Concentra los atributos identificativos (email único, par username + tag también único, modelo inspirado en plataformas tipo Discord) y las credenciales hasheadas. Es el sujeto de la autenticación y, por tanto, el origen de cualquier acción auditable.

La tabla **grupos** es el contenedor lógico de contenido y de permisos. Un grupo agrupa vídeos y usuarios, y define el ámbito de visibilidad para esos vídeos. Mantiene siempre una referencia obligatoria al usuario creador (id_creador) con ON DELETE RESTRICT para evitar que se elimine accidentalmente un usuario que todavía es propietario de grupos.

La tabla **videos** representa el activo audiovisual. Contiene los metadatos (título, duración, nombre de fichero, miniatura, fecha de subida) y mantiene una referencia al usuario propietario (usuario_id con ON DELETE SET NULL: si se borra el usuario, el vídeo permanece pero pierde la atribución). El archivo binario propiamente dicho se almacena fuera de la base de datos, en el sistema de ficheros del servidor —directorio uploads/ montado como volumen— y la base de datos guarda únicamente el nombre.

La tabla **miembros_grupo** es la tabla de unión que relaciona usuarios y grupos. Se modela con **clave primaria compuesta** (id_usuario, id_grupo) porque la relación se entiende como un hecho atómico: una persona forma parte —o no— de un grupo, sin necesidad de un identificador artificial. Esta tabla almacena, además, el rol del usuario dentro del grupo (VARCHAR(20), con valor por defecto *MIEMBRO*) y la fecha_union. Ambas claves foráneas usan ON DELETE CASCADE: si se borra un usuario o un grupo, su pertenencia desaparece con él.

La tabla **permisos_grupo** resuelve la relación N:M entre vídeos y grupos: qué vídeo es visible para qué grupo. La clave primaria es de nuevo compuesta (id_video, id_grupo) y ambas FK utilizan ON DELETE CASCADE. Este diseño permite que un mismo vídeo se publique en varios grupos sin duplicar el fichero físico ni los metadatos.

La capa analítica se estructura en torno a dos tablas con responsabilidades diferenciadas y complementarias, lo que merece especial atención porque suele ser un patrón confuso si no se explica con claridad.

La tabla **video_vistas** funciona como un contador agregado por par usuario+vídeo. Su restricción UNIQUE (id_usuario, id_video) garantiza que existe a lo sumo una fila para cada combinación, y el campo contador se actualiza mediante un patrón de tipo UPSERT cada vez que un usuario inicia una visualización efectiva. Esta tabla responde de forma extremadamente eficiente a preguntas como «¿cuántas veces ha visto este vídeo este usuario?» o «¿qué vídeos son los más reproducidos?», sin necesidad de agregar filas de eventos.

La tabla **logs**, en cambio, almacena telemetría detallada por sesión de reproducción. Cada fila contiene el usuario, el vídeo, el grupo desde el que se accedió (id_grupo es *nullable* porque un usuario puede reproducir vídeos propios fuera de cualquier grupo), la ip_acceso, el session_id que identifica unívocamente una sesión de reproducción y, sobre todo, el campo segundos_vistos. El flujo es el siguiente: cuando el reproductor inicia una sesión, emite latidos periódicos (*heartbeats*) hacia el backend, que se aplican como UPSERT sobre la fila identificada por la cuádruple (id_usuario, id_video, id_grupo, session_id). La restricción UNIQUE sobre esa cuaterna garantiza que cada sesión acumula su tiempo en una única fila, en lugar de generar miles de eventos por reproducción.

Esta combinación de un contador agregado (*video_vistas*) y una telemetría granular por sesión (*logs*) es una decisión deliberada: *video_vistas* responde con latencia mínima a las preguntas más frecuentes del panel de analíticas, mientras que *logs* permite calcular métricas más sofisticadas (duración media efectiva, % de completado, sesiones por día) cuando se requieren.

Sobre la convención de nomenclatura merece la pena hacer una nota: las clases Java del dominio se han escrito en inglés y en PascalCase, siguiendo las convenciones idiomáticas del lenguaje, mientras que los nombres físicos de tablas y columnas en MySQL se han mantenido en español (usuarios, grupos, miembros_grupo, permisos_grupo, videos, video_vistas, logs). Es una decisión consciente que prioriza la legibilidad del SQL para personas no técnicas que puedan acceder directamente a la base de datos, sin renunciar a la coherencia del código.

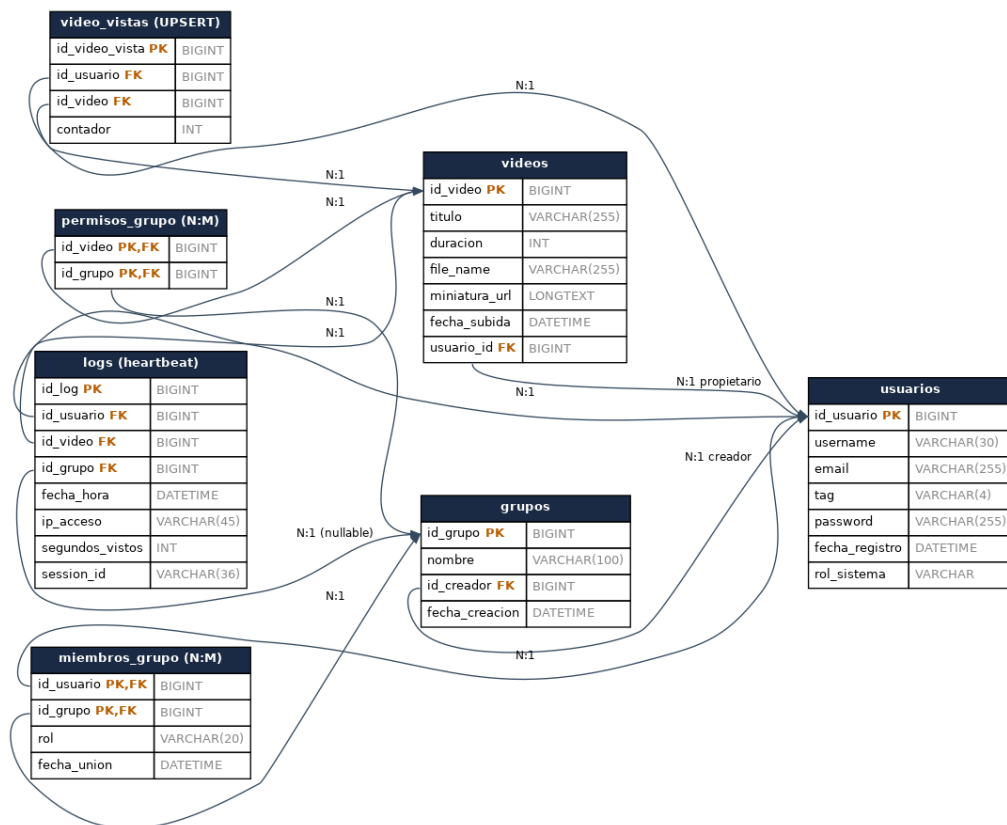


Figura 1. Diagrama entidad-relación de Lockstrm, derivado del DDL real (V1__init.sql).

6.3. Esquema de funcionamiento de la aplicación

El flujo de una sesión típica en Lockstrm permite ver con claridad cómo encajan todas las piezas tecnológicas. El recorrido empieza en el navegador y termina en el contenedor del backend, atravesando el proxy inverso, el filtro de autenticación y, finalmente, el controlador que sirve el flujo de vídeo y registra el consumo.

Cuando el usuario abre Lockstrm en su navegador, recibe los artefactos estáticos del cliente Angular servidos por Nginx. Una vez cargada la aplicación, el formulario de inicio de sesión captura las credenciales y las envía contra el endpoint POST /api/auth/login del backend. Este endpoint está sujeto al *RateLimitFilter*, que limita las peticiones a 10 por minuto y por dirección IP. Esta limitación, aparentemente simple, es una primera línea de defensa contra ataques de fuerza bruta sin requerir infraestructura adicional.

Si las credenciales son válidas, el backend genera un JSON Web Token firmado con la biblioteca JWT, con una caducidad de ocho horas y un *payload* que incluye el identificador del usuario y sus roles efectivos. El token se devuelve al cliente, que lo almacena en memoria —no en *localStorage* para reducir la superficie de ataque XSS— y lo adjunta como cabecera Authorization: Bearer <token> en cada llamada posterior.

A partir de ese momento, el usuario navega por la aplicación. Cada vista de Angular se implementa como un Standalone Component que utiliza *Signals* para gestionar su estado local. Las peticiones HTTP pasan por un interceptor que inyecta automáticamente la cabecera Authorization, y por un segundo interceptor que detecta respuestas 401 y redirige

al login. En el backend, esas peticiones son procesadas por la cadena de filtros de Spring Security: primero el filtro JWT valida el token y construye el contexto de seguridad, y a continuación las anotaciones de autorización a nivel de método (`@PreAuthorize`) garantizan que el rol del usuario es suficiente para la operación solicitada.

El caso del streaming de vídeo merece un tratamiento aparte. Los reproductores HTML5 nativos no permiten establecer cabeceras personalizadas en la petición que generan al cargar la etiqueta `<video src>`, y el reproductor utiliza peticiones HTTP de tipo *Range* (código 206) para descargar fragmentos del vídeo a medida que el usuario avanza por la línea de tiempo. Esto crea un conflicto: no podemos inyectar la cabecera `Authorization` en esas peticiones desde Angular como hacemos con el resto del API.

La solución adoptada en Lockstrm pasa por un **Service Worker** registrado en el cliente. Este `Service Worker` intercepta todas las peticiones salientes hacia el endpoint de streaming, comprueba si el token está disponible y, si lo está, reescribe la URL añadiendo el token como *query parameter* (`?token=...`). En el backend, un filtro específico para esa ruta acepta el token tanto por cabecera como por *query string*, lo valida y autoriza la petición. Una vez autorizado, el controlador atiende la petición *Range* devolviendo el fragmento del archivo solicitado con código 206 y las cabeceras `Content-Range` y `Accept-Ranges` adecuadas.

En paralelo, mientras el vídeo se reproduce, el cliente emite latidos periódicos (cada pocos segundos) al endpoint `POST /api/logs/heartbeat`. Cada latido lleva el `session_id` generado al iniciar la reproducción y el número de segundos acumulados; el servicio `LogService.registrarHeartbeat()` aplica un `UPSERT` sobre la tabla `logs` de modo que cada sesión vive en una sola fila que se va actualizando. La primera reproducción efectiva, además, dispara la operación `UPSERT` sobre `video_vistas` para incrementar el contador del par usuario-vídeo.

Toda esta cadena queda detrás de Nginx, que actúa como punto único de entrada, termina la conexión TLS, sirve los activos estáticos del frontend y reenvía las peticiones de la API y del streaming al contenedor del backend. En paralelo, el endpoint `/actuador/prometheus` queda expuesto para que Prometheus consuma métricas de uso (peticiones por segundo, latencia, estado del *pool* de conexiones JDBC, tiempo de respuesta del streaming) y permita diagnosticar problemas en producción.

6.4. Descomposición modular e interrelación

Lockstrm se organiza como un sistema desacoplado en dos grandes piezas comunicadas por una API HTTP: el backend, responsable de toda la lógica de negocio y de la persistencia, y el frontend, responsable de la interacción con el usuario. Esta separación no es accidental: permite que ambos lados evolucionen a ritmos distintos y abre la puerta a que, en el futuro, otros clientes —una app móvil nativa, un dashboard de administración separado, una integración con un SSO corporativo— consuman la misma API sin modificarla.

En el backend, la arquitectura sigue una organización por capas. La capa de controladores (@RestController) expone los *endpoints* REST y se limita a delegar; la capa de servicios concentra la lógica de negocio y orquesta las interacciones entre repositorios; la capa de persistencia, basada en Spring Data JPA, traduce las entidades del dominio a filas en MySQL. Por encima de todas ellas, los filtros de seguridad y los *aspect oriented* de Spring proporcionan transversalidad: autenticación, autorización, registro de eventos.

El frontend, por su parte, está organizado en torno a *features*: cada área funcional —autenticación, gestión de grupos, reproducción de vídeo, panel de analíticas, ajustes— vive en su propia carpeta y agrupa los componentes, servicios y rutas que le son propios. Esta topología por *features*, combinada con los Standalone Components, evita los problemas clásicos de los proyectos Angular antiguos basados en módulos compartidos crecientes.

Toda la infraestructura se despliega con Docker Compose. El archivo docker-compose.yml define los servicios y sus relaciones: el contenedor *mysql* aporta la base de datos y expone únicamente la red interna, el contenedor *backend* ejecuta el JAR de Spring Boot, el contenedor *nginx* publica los puertos 80/443 y enruta el tráfico, y el contenedor *prometheus* (opcional en desarrollo) consume las métricas. La red Docker queda aislada del exterior salvo por el punto de entrada de Nginx, lo que reduce la superficie de ataque.

6.5. Descripción de los módulos

Aunque el código del backend no se estructura formalmente en módulos Java independientes —se trata de un único proyecto Spring Boot con paquetes—, conceptualmente se pueden identificar tres bloques funcionales claramente diferenciados.

Módulo de Seguridad y Autenticación

Es el módulo transversal por excelencia. Se ocupa de la autenticación de usuarios mediante credenciales, de la emisión y validación de JWT, y de la aplicación de las reglas de autorización por rol en cada petición. Sus piezas clave son: el *UserDetailsService* personalizado que carga el usuario desde la base de datos, el *JwtAuthenticationFilter* que extrae y valida el token en cada petición, el *RateLimitFilter* que protege los endpoints de autenticación frente a fuerza bruta y los @PreAuthorize declarativos repartidos por los controladores. La caducidad de 8 horas en los tokens es un equilibrio razonable entre seguridad y experiencia de usuario: lo bastante corta para limitar el impacto de un token comprometido, y lo bastante larga para que un usuario no tenga que volver a autenticarse en una jornada laboral típica. Una pieza adicional, fruto de la migración V6__password_reset_tokens.sql, gestiona los tokens de recuperación de contraseña con caducidad propia.

Módulo de Gestión de Grupos y Roles

Este módulo encapsula toda la lógica relativa a la organización de usuarios y permisos. Es el responsable de crear grupos, añadir y eliminar miembros, asignar roles dentro de cada grupo y comprobar si un usuario tiene autoridad suficiente para una acción concreta. La jerarquía *SUPER_ADMIN* → *ADMIN* → *EDITOR* → *MIEMBRO* se materializa en las reglas

de negocio que viven aquí: el *SUPER_ADMIN* tiene capacidad total sobre la plataforma; el *ADMIN* gobierna un grupo concreto y puede invitar, expulsar y promover miembros; el *EDITOR* puede subir y editar contenido pero no modificar la composición del grupo; el *MIEMBRO* únicamente consume contenido.

Módulo de Streaming y Analíticas

Es el corazón funcional de Lockstrm. Por un lado, gestiona la subida y almacenamiento de vídeos, su asignación a grupos y la entrega de los archivos a los clientes mediante peticiones HTTP por rangos. Por otro, alimenta dos canales analíticos complementarios: un contador agregado en *video_vistas*, que registra cada visualización iniciada por el par usuario-vídeo, y una bitácora de heartbeats en logs, que acumula tiempo efectivamente reproducido por sesión y desde qué grupo se consumió el contenido. El módulo es también donde se concentra la mayor sutileza técnica del proyecto: el manejo de *range requests* en Spring Boot, la integración con el Service Worker del cliente para la inyección del token vía *query string* y el patrón UPSERT idempotente que mantiene las dos tablas analíticas coherentes sin saturar la base de datos.

6.6. Descripción de la interfaz de usuario

La interfaz de Lockstrm se ha diseñado bajo una filosofía deliberadamente sobria: ofrecer al usuario lo que necesita en cada momento, sin elementos decorativos que distraigan, y manteniendo una jerarquía visual clara entre las áreas de la aplicación. La intención no es deslumbrar con efectos, sino que cualquier persona con un mínimo de familiaridad con plataformas de vídeo encuentre el camino al contenido en pocos clics.

Tecnológicamente, esta filosofía se materializa en Angular 21 mediante el uso intensivo de **Standalone Components**. Cada vista (login, listado de grupos, detalle de grupo, reproductor, panel de analíticas, ajustes) es un componente autónomo que declara explícitamente sus dependencias en el campo `imports`. Esta autonomía elimina la necesidad de los NgModules tradicionales, reduce el ruido conceptual y produce *bundles* más pequeños porque el compilador puede aplicar *tree-shaking* con mayor agresividad. Durante el desarrollo se han ido construyendo además componentes transversales propios —en particular un `CustomSelectComponent` que reemplaza al elemento `<select>` nativo en toda la aplicación— para garantizar un aspecto coherente independientemente del navegador del usuario.

El estado interno de cada componente se gestiona con **Signals**, el nuevo modelo de reactividad introducido en Angular. En lugar de la mezcla histórica de propiedades, *BehaviorSubjects* de RxJS y *ChangeDetection* manual, las Signals ofrecen un modelo declarativo y trazable: una variable es una signal, su consumidor (plantilla o computed) se re-evalúa automáticamente al cambiar el valor, y el ciclo de detección de cambios se restringe a las dependencias reales. El resultado es un frontend más predecible, con menos efectos secundarios y, en igualdad de condiciones, más eficiente en runtime.

Las rutas del cliente —siguiendo la convención del proyecto— están en español (`/login`, `/grupos`, `/grupos/:id`, `/ajustes`, `/analiticas`), lo que hace que la URL sea inmediatamente

comprensible para el usuario final. El paralelismo con las rutas del API (`/api/grupos`, `/api/analiticas`) no es casualidad: mantener una nomenclatura consistente reduce la fricción cognitiva cuando se diagnostican problemas.

6.7. Descripción de listados e informes

La capa analítica de Lockstrm se construye sobre las dos tablas dedicadas, *video_vistas* y *logs*, y se expone a través del *endpoint* `GET /api/analiticas` y sus variantes parametrizadas. Su objetivo es responder a las preguntas que típicamente plantean los administradores de la plataforma: cuántas visualizaciones acumula cada vídeo, qué grupos son los más activos, qué usuarios concretos han visto un vídeo determinado y cuál es la evolución del consumo a lo largo del tiempo.

Las consultas se reparten entre las dos tablas analíticas según el tipo de pregunta. Cuando lo que se necesita es un recuento agregado —ranking de los vídeos más vistos, número total de espectadores únicos de un vídeo, lista de vídeos que un usuario ha visto al menos una vez— se utiliza *video_vistas*, con consultas que se resuelven con simples `COUNT` y `GROUP BY`. Cuando lo que se requiere es información temporal o de profundidad de consumo —segundos vistos por usuario, distribución temporal de las reproducciones, sesiones desde un grupo concreto— se consulta *logs*, que aporta la dimensión *session_id* y el campo *segundos_vistos*.

El *endpoint* `/api/analiticas` implementa estos agregados como proyecciones tipadas de Spring Data, evitando el uso de SQL en bruto cuando es posible y centralizando las consultas más complejas en repositorios dedicados. La respuesta se entrega en formato JSON estructurado, listo para ser representado en gráficos en el cliente. Para la renderización visual, en el frontend se han empleado componentes de gráficos sencillos basados en SVG: histogramas para la evolución temporal, listados ordenados para los rankings y tarjetas resumen para los indicadores clave.

Conviene subrayar una decisión de diseño concreta: la analítica se calcula *on demand* —en el momento de la petición— en lugar de precomputarse y almacenarse en una tabla de resumen. El patrón de UPSERT sobre *video_vistas* ya elimina la mayor parte del coste —no hay que recorrer eventos para contar visualizaciones, basta con leer el contador— y los índices definidos sobre *logs* (*idx_logs_grupo_video*, *idx_logs_grupo_fecha*) hacen que las consultas temporales sean razonablemente rápidas en órdenes de magnitud actuales. Si en el futuro Lockstrm escalara a millones de sesiones, sería natural introducir una capa de agregación periódica sin que ello altere la interfaz pública del *endpoint*.

6.8. Manual de usuario

El manual de usuario tiene como objetivo permitir que cualquier persona con un rol asignado en Lockstrm pueda utilizar la plataforma sin necesidad de formación técnica previa. A modo de guion general, las operaciones cubiertas son las siguientes:

- **Registro e inicio de sesión:** el usuario accede a la URL de Lockstrm, se registra indicando username, tag, correo y contraseña, y a partir de ahí se autentica con sus

credenciales para obtener una sesión de 8 horas.

- **Recuperación de contraseña:** en caso de olvido, el flujo de *password reset* emite un token de un solo uso con caducidad propia que permite establecer una nueva contraseña.
- **Navegación por grupos:** tras autenticarse, el usuario ve la lista de grupos a los que pertenece. Al entrar en un grupo, accede al catálogo de vídeos disponibles.
- **Reproducción:** al seleccionar un vídeo, el reproductor se carga con soporte de búsqueda en la línea de tiempo y la visualización se registra automáticamente (contador agregado y latidos de sesión).
- **Administración de grupo (rol ADMIN):** permite invitar nuevos miembros, asignar roles dentro del grupo y eliminar miembros que ya no deban tener acceso.
- **Subida y edición de vídeos (rol EDITOR o superior):** permite añadir contenido al grupo, editar metadatos, subir miniaturas y eliminar vídeos obsoletos.
- **Panel de analíticas:** disponible para administradores, muestra el consumo agregado de cada vídeo y de cada grupo, así como rankings de usuarios y evolución temporal.
- **Ajustes personales:** cada usuario puede actualizar sus datos básicos, su avatar y cambiar su contraseña desde /ajustes.

[STAND-BY: Completar por el alumno] Ampliar este apartado con capturas de pantalla anotadas de cada flujo. Recomendable un par de pantallazos por flujo (formulario y resultado) con leyendas breves; conviene tomar las capturas tanto en versión escritorio como móvil, dado que el frontend se rediseñó para responsive en los commits del 6 de mayo.

6.9. Manual de instalación o despliegue

El despliegue de Lockstrm se ha pensado para que sea reproducible en cualquier entorno con Docker disponible. La idea rectora es que un evaluador o un administrador pueda levantar la plataforma completa sin instalar Java, Maven, Node, MySQL ni Nginx en su máquina anfitriona, y obtener exactamente el mismo comportamiento que en producción.

El procedimiento manual consta de tres pasos. En primer lugar, se clona el repositorio del proyecto. En segundo lugar, se crea un fichero `.env` a partir del fichero `.env.example` incluido en el repositorio; este fichero contiene las variables sensibles de la configuración: secreto del JWT, usuario y contraseña de la base de datos, puertos expuestos y rutas de almacenamiento de vídeos. En tercer lugar, se ejecuta el comando `docker compose up -d --build`, que construye las imágenes del backend y del frontend si es necesario y levanta los contenedores en segundo plano.

Para entornos de producción, el repositorio incluye además un *workflow* de despliegue automático en `.github/workflows/deploy.yml`. Cada *push* a la rama `main` dispara el siguiente flujo: GitHub Actions arranca un *runner* Ubuntu, configura una clave SSH a partir del secreto `SSH_PRIVATE_KEY`, se conecta al servidor de producción definido en `SERVER_HOST` y ejecuta sobre él `git pull origin main`, `docker compose up -d --build`

--remove-orphans y docker image prune -f. El resultado es un despliegue continuo sin intervención manual y sin sorpresas: lo que pasa la verificación en GitHub es exactamente lo que corre en producción.

Una vez levantado el sistema, la aplicación está disponible en la URL configurada en Nginx (por defecto `http://localhost`; en producción, la URL del dominio configurado). El primer usuario administrador puede crearse mediante un *seed* SQL incluido en el repositorio o mediante una llamada controlada al endpoint de registro, según se haya parametrizado el entorno. A partir de ahí, todo el ciclo administrativo se realiza desde la propia interfaz web.

Para detener la plataforma basta con `docker compose down`; los datos de la base de datos persisten en un volumen Docker dedicado, de modo que el contenido sobrevive a los reinicios. Las copias de seguridad pueden realizarse exportando ese volumen y, complementariamente, ejecutando `mysqldump` contra el contenedor de la base de datos.

7. Planificación del proyecto

La planificación de Lockstrm se ha articulado en fases solapadas, asumiendo desde el inicio que un trabajo que combina backend, frontend e infraestructura no admite una planificación rígida en cascada: las decisiones de cada capa repercuten en las demás y obligan a iterar. El cronograma real, reconstruido a partir del historial del repositorio (114 commits entre diciembre de 2025 y mayo de 2026), refleja esta estructura.

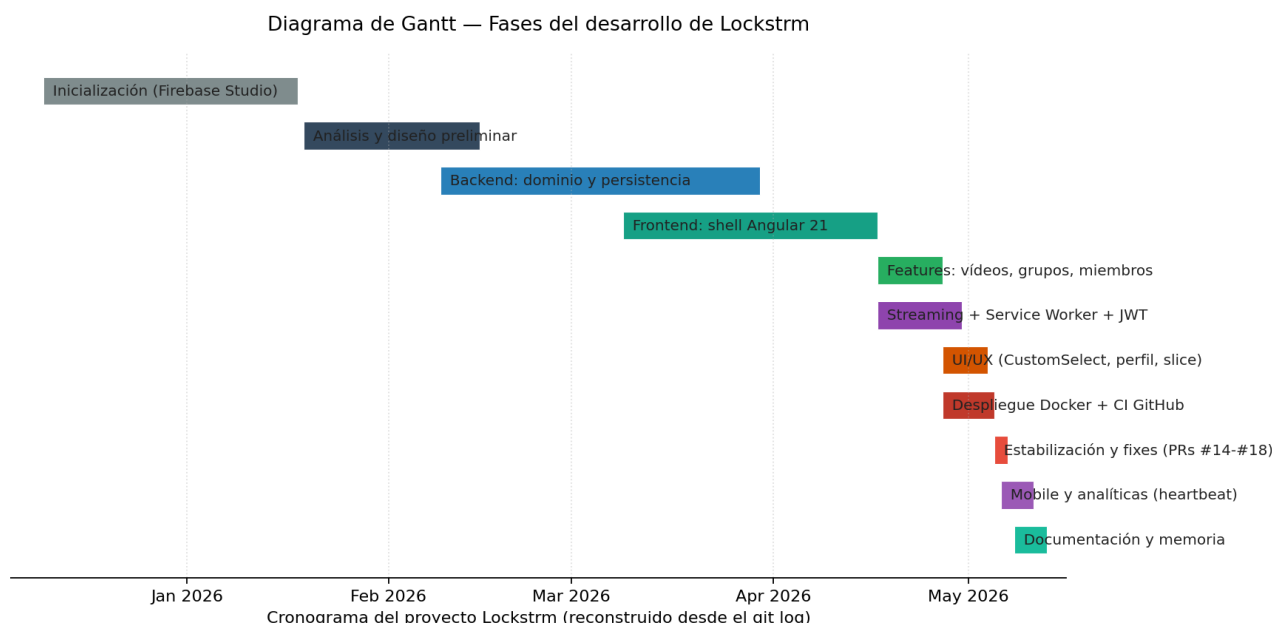


Figura 2. Diagrama de Gantt reconstruido a partir del historial de commits del repositorio (rango 2025-12-10 → 2026-05-11).

Descomposición en fases (WBS)

El proyecto se descompone en once fases que cubren el ciclo de vida completo del desarrollo:

- **F1. Inicialización (Dic 2025):** creación del workspace en Firebase Studio, decisión de stack y bootstrapping del proyecto.
- **F2. Análisis y diseño preliminar (Ene 2026):** definición del modelo de dominio y primeras pruebas con Angular.
- **F3. Backend — dominio y persistencia (Feb-Mar 2026):** entidades JPA, repositorios, migraciones Flyway iniciales (V1-V5), capa de servicios.
- **F4. Frontend — shell de Angular 21 (Mar-Abr 2026):** rutas, layouts, autenticación, interceptores HTTP, primer reproductor.
- **F5. Features principales (Abr 2026):** vídeos en grupos, miniaturas, gestión de miembros, primeras estadísticas.
- **F6. Streaming + Service Worker + JWT (Abr 2026):** integración del Service Worker con el token, filtro de seguridad *dual* en el backend, manejo de *range requests*.

- **F7. UI/UX consolidada (finales de Abr 2026):** CustomSelectComponent, *pill chips* para selector de grupos, perfil con avatar, cambios de estilo.
- **F8. Despliegue Docker + CI (Abr-May 2026):** docker-compose.yml definitivo, workflow de GitHub Actions con SSH, configuración de Nginx.
- **F9. Estabilización (5-7 May 2026):** ronda de PRs #14-#18 centrados en bugs descubiertos al desplegar (Hibernate enum varchar, codificación de la clave SSH, exposición de puertos).
- **F10. Adaptación móvil y analíticas finales (6-11 May 2026):** diseño responsive, panel de analíticas con consumo por *heartbeats*.
- **F11. Documentación y memoria (May 2026):** redacción de la presente memoria y preparación de la defensa.

Recursos

El proyecto ha sido desarrollado por una sola persona, sin equipo adicional. Los recursos materiales empleados son los habituales de un entorno de desarrollo web: equipo personal con IDE (IntelliJ IDEA para backend y VS Code para frontend), repositorio en GitHub para el control de versiones y CI, y un servidor de producción para el despliegue automatizado vía SSH. Las dependencias software son todas de licencia abierta (Java OpenJDK, MySQL Community, Spring Boot, Angular, Nginx, Docker).

Prevención de riesgos

Los principales riesgos identificados al inicio del proyecto y las medidas mitigadoras aplicadas han sido:

- **Riesgo:** incompatibilidades entre versiones recientes de Spring Boot 3 y Hibernate 6 con MySQL. **Mitigación:** fijar versiones LTS y migraciones Flyway versionadas que documenten cada cambio de esquema (este riesgo se materializó parcialmente — ver apartado 9).
- **Riesgo:** pérdida de productividad por entornos de desarrollo divergentes. **Mitigación:** contenedorización completa con docker-compose desde fase temprana.
- **Riesgo:** bloqueo por incidente con el repositorio o el servidor. **Mitigación:** repositorio público en GitHub, despliegue idempotente y reproducible.
- **Riesgo:** alcance excesivo respecto al tiempo disponible. **Mitigación:** recortar funcionalidades secundarias como transcoding adaptativo, DRM o SSO, dejándolas explícitamente como mejoras futuras (ver apartado 10).

[STAND-BY: Completar por el alumno] Si la normativa del centro exige fechas de inicio y fin contractuales (las pactadas con el tutor) distintas de las fechas reales del repositorio, sustituirlas en el Gantt antes de la entrega. El cronograma actual refleja la ejecución real, no el plan original.

8. Control de la ejecución

El control de la ejecución de Lockstrm se ha sustentado, ante todo, en mecanismos cualitativos y artesanales, antes que en una suite automatizada de pruebas. Esto es honesto reconocerlo y, además, es congruente con la realidad de muchos proyectos individuales: el tiempo disponible obligó a priorizar el desarrollo de funcionalidad sobre la inversión en infraestructura de testing automatizado, sin que ello implique que la calidad haya quedado desatendida.

Indicadores empleados

- **Revisión por *pull requests*:** los cambios de mayor riesgo se han integrado a través de PRs en GitHub (existen al menos los PRs #11, #14, #15, #17, #18), lo que ha permitido revisar cada lote de cambios fuera del flujo principal antes de fusionarlo en main.
- **Verificación manual sobre entorno desplegado:** el workflow de GitHub Actions redespiega el sistema completo en cada *push* a main, lo que ha convertido la propia producción en un entorno de validación continua.
- **Pruebas exploratorias guiadas:** tras cada bloque funcional —subida de vídeos, gestión de miembros, reproducción con *range requests*, panel de analíticas— se han realizado pruebas manuales recorriendo los flujos completos en distintos navegadores y dispositivos, incluida la versión móvil.
- **Migraciones versionadas:** Flyway ha actuado como red de seguridad para el esquema; cualquier cambio en la base de datos ha pasado por una migración numerada (V1__init a V9__normalize_users) que documenta y replica el cambio de forma determinista en cualquier entorno.
- **Plantilla mínima de tests:** el proyecto incluye una clase LockstrmApplicationTests con la prueba contextLoads() que garantiza, en cada compilación, que el contexto de Spring puede arrancar sin errores; es el primer eslabón del que partir para añadir tests de servicio y de controlador en futuras iteraciones.

Criterios de evaluación

Los criterios cualitativos aplicados durante el desarrollo han sido: que cada PR *compile* y arranque limpiamente en local; que los flujos críticos —login, listado de grupos, reproducción, latidos de analíticas— funcionen tras cada cambio; que las migraciones de base de datos puedan aplicarse desde una base vacía sin intervención manual; y que el despliegue continuo termine sin errores.

[STAND-BY: Completar por el alumno] Si el tribunal o la normativa exigen indicadores cuantitativos formales —porcentaje de cobertura de código, número de defectos abiertos/cerrados, lead time, MTTR—, conviene indicarlo aquí explícitamente. La mejora natural en una próxima iteración es introducir JaCoCo para cobertura, ampliar la suite de tests JUnit a la capa de servicios y añadir tests E2E con Playwright sobre el frontend; este paso queda recogido en el apartado 10 como propuesta de mejora.

9. Dificultades encontradas

Más allá de las dificultades habituales en cualquier proyecto de software —comprensión del dominio, depuración de errores, gestión del tiempo—, el desarrollo de Lockstrm ha planteado una serie de retos técnicos concretos que vale la pena documentar, porque condicionaron decisiones de diseño importantes y aparecen claramente reflejados en el historial del repositorio.

Gestión de *HTTP Range Requests* en Spring Boot. Servir vídeo a través de una etiqueta `<video>` de HTML5 no es lo mismo que servir un fichero estático: el reproductor del navegador envía peticiones con cabecera *Range* para descargar fragmentos del archivo a medida que el usuario navega por la línea de tiempo o se adelanta en la reproducción. El servidor debe responder con código 206 (*Partial Content*) y las cabeceras *Content-Range* y *Accept-Ranges* adecuadas. Implementar esto correctamente en Spring Boot exige interpretar el rango solicitado, validar sus límites, abrir un *RandomAccessFile* sobre el archivo físico y devolver únicamente la porción solicitada. La primera versión del módulo de streaming sufrió varios casos límite —rangos abiertos, rangos invertidos, rangos fuera del tamaño del archivo— que sólo afloraron al probar con reproductores reales.

Inyección del JWT en peticiones de vídeo mediante Service Worker. Como se ha explicado en el apartado de funcionamiento, los reproductores HTML5 no permiten personalizar las cabeceras de las peticiones *Range*. Esto chocaba con el modelo de autenticación basado en cabecera *Authorization: Bearer* que se aplica al resto de la API. Las opciones evaluadas —usar *cookies* con *SameSite* estricto, exponer URLs firmadas de un solo uso o transmitir el token por *query*— tenían cada una sus inconvenientes. La solución finalmente adoptada combina lo mejor de dos enfoques: el cliente sigue almacenando el JWT en memoria, pero un Service Worker intercepta las peticiones hacia el endpoint de streaming y reescribe la URL añadiendo el token como *query parameter*. Esto exigió, además, parametrizar el filtro de seguridad del backend para aceptar el token por ambas vías exclusivamente en esa ruta, manteniendo el resto del API estricto.

Migración mental del estado a *Signals* en Angular. Trabajar con *Signals* supone renunciar a hábitos arraigados de programación reactiva con RxJS. Si bien la API es sencilla, la dificultad real está en repensar cómo se compone el estado: cuándo conviene una *signal* simple, cuándo una *computed* derivada y cuándo es todavía preferible un *Observable* para flujos asíncronos. En las primeras iteraciones del frontend hubo casos en los que se mezclaron ambos modelos sin una regla clara, produciendo componentes difíciles de razonar. Adoptar la convención de mantener el estado local en *Signals* y reservar RxJS para los efectos asíncronos puntuales —principalmente llamadas HTTP— resolvió el problema.

Diseño analítico dual: contador agregado y telemetría por sesión. Inicialmente la analítica se modeló como una única tabla de eventos, una fila por reproducción iniciada. Ese diseño daba problemas al responder preguntas frecuentes (*rankings*, número de espectadores únicos) que obligaban a recorrer la tabla completa. La solución fue desdoblar: un contador agregado en *video_vistas* con *UNIQUE* sobre el par usuario-vídeo, gestionado vía *UPSERT*, y una bitácora de *heartbeats* en logs con *UNIQUE* sobre la cuádruple (usuario, vídeo, grupo, sesión), también *UPSERT*. Hacer convivir ambas tablas y

mantenerlas coherentes obligó a entender bien la semántica de `INSERT ... ON DUPLICATE KEY UPDATE` y el comportamiento de MySQL ante NULL en columnas UNIQUE (que no se considera duplicado), problema que terminó resolviéndose en lógica de servicio.

Incompatibilidades de Hibernate 6 con enums MySQL. Al desplegar por primera vez contra producción se descubrió que el mapeo automático de `@Enumerated(EnumType.STRING)` generaba columnas con longitud insuficiente, lo que rompía la inserción de roles. La solución fue una migración correctiva (`V4__fix_group_role_varchar.sql`) que fuerza `VARCHAR(20)` y un commit específico (*fix: force varchar(20) for GroupRole enum column (Hibernate 6 MySQL compat)*) para alinear la entidad.

Pipeline de despliegue: claves SSH y saltos de línea. Configurar el workflow de GitHub Actions para hacer SSH contra el servidor de producción reveló un problema sutil: las claves SSH almacenadas en los secretos de GitHub perdían los saltos de línea al ser inyectadas como variables de entorno. La cadena de commits *ci: test automatic deploy* → *retest* → *use env var for SSH key to preserve newlines* → *use base64-encoded SSH key to fix newline issue* → *strip whitespace before base64 decode* es el rastro fiel de varias hipótesis hasta dar con la solución definitiva: codificar la clave en Base64 antes de almacenarla como secreto y decodificarla en el *runner*. Es una dificultad menor pero ilustrativa de cuánto puede consumir un detalle aparentemente trivial cuando bloquea el despliegue.

Exposición de puertos en el contenedor del frontend. Otro fallo descubierto en despliegue: el contenedor del frontend no estaba publicando correctamente el puerto 80, lo que dejaba la aplicación inaccesible incluso con Nginx funcionando. Su solución vino acompañada de la adición del volumen de avatares, una incidencia recogida en el commit *fix: expose frontend on port 80, add avatars volume*.

[STAND-BY: Completar por el alumno] Añadir aquí las dificultades personales y de gestión del tiempo experimentadas durante el desarrollo: conciliación con asignaturas paralelas, periodos de bloqueo creativo o técnico, problemas de equipamiento, decisiones de alcance que tuvieron que reescalarsse, etc. Estas dificultades, aunque no sean estrictamente técnicas, forman parte legítima del aprendizaje del TFG y suelen ser muy valoradas por el tribunal cuando se cuentan con honestidad.

10. Propuestas de mejora

El estado actual de Lockstrm cubre los requisitos funcionales para los que fue concebido, pero hay varias líneas de evolución que ampliarían su valor o lo prepararían para escenarios más exigentes. Se enumeran a continuación, ordenadas por nivel de impacto técnico.

Suite de pruebas automatizadas. Es, por las razones expuestas en el apartado 8, la mejora más rentable a corto plazo. Implica ampliar la cobertura de tests unitarios sobre la

capa de servicios, añadir tests de integración con Testcontainers (MySQL real en cada ejecución de la CI) y construir una suite de tests end-to-end con Playwright sobre el frontend. Esta mejora no aporta funcionalidad nueva, pero protege el proyecto frente a regresiones cuando se acometan las mejoras siguientes.

Caché distribuida con Redis. Los metadatos de los vídeos, las pertenencias a grupos y las reglas de permisos se consultan en prácticamente cada petición autenticada. Introducir una caché distribuida como Redis para estos accesos reduciría drásticamente la carga sobre MySQL y disminuiría la latencia percibida por el usuario. Spring Cache se integra con Redis prácticamente sin código.

Almacenamiento de vídeo en *object storage*. Hoy los archivos viven en el sistema de ficheros del contenedor del backend, lo que limita la escalabilidad horizontal y obliga a sincronizar volúmenes si se quisiera tener más de una réplica. Migrar el almacenamiento a un servicio compatible con S3 (Amazon S3, MinIO autoalojable, Backblaze B2) desacoplaría el backend del almacenamiento y permitiría servir vídeo desde una CDN, mejorando el rendimiento global.

Escalado horizontal del backend. El diseño *stateless* con JWT permite, en principio, levantar varias instancias del backend y repartir la carga con Nginx en modo balanceador. Las únicas piezas que requieren coordinación —el *RateLimitFilter* y, en su caso, un sistema de revocación de tokens— pueden apoyarse en Redis. Esta evolución es natural y de bajo coste arquitectónico.

Transcoding adaptativo y entrega vía HLS o DASH. Servir el archivo original tal cual obliga al cliente a descargar la calidad máxima disponible. Introducir un pipeline de transcoding (con FFmpeg) que produzca varias versiones del vídeo a distintos *bitrates* y entregarlas mediante HLS o MPEG-DASH permitiría al reproductor elegir la calidad adecuada al ancho de banda en cada momento. Es la evolución técnica más ambiciosa, pero también la que más aproximaría Lockstrm a la experiencia de las plataformas comerciales.

Marcas de agua dinámicas y refuerzo del DRM. Para escenarios donde la trazabilidad de la copia es crítica, podría añadirse una marca de agua superpuesta con el identificador del usuario, así como integración con sistemas DRM estándar (Widevine, FairPlay).

Integración con SSO corporativo. Soportar SAML 2.0 u OpenID Connect permitiría que Lockstrm se enchufara a los *identity providers* habituales en empresas (Azure AD, Okta, Keycloak), eliminando la necesidad de gestionar credenciales propias y facilitando su adopción corporativa.

Observabilidad ampliada. Sumar a Prometheus una capa de *tracing* distribuido (OpenTelemetry → Jaeger o Tempo) permitiría diagnosticar problemas de latencia que hoy son difíciles de aislar, especialmente en escenarios de streaming concurrente.

Pre-agregación periódica de analíticas. A medida que logs crezca, convendrá introducir una tabla `logs_diarios` alimentada por una tarea programada que agregue por día, vídeo y grupo. La interfaz pública del endpoint `/api/analiticas` no necesita cambiar; sólo cambia el

origen de los datos para las consultas históricas.

11. Conclusión final

El desarrollo de Lockstrm parte de una premisa sencilla y a la vez exigente: demostrar que es posible construir, en el marco de un Trabajo de Fin de Grado, una plataforma B2B de streaming privado que combine seguridad, control granular del acceso y observabilidad del consumo, utilizando un stack tecnológico moderno y plenamente actual.

El resultado, valorado en conjunto, cumple ese objetivo. El backend en Spring Boot 3 sobre Java 21 ha demostrado ser una base sólida y productiva, capaz de albergar tanto la lógica transaccional de la gestión de grupos y roles como la lógica más exigente del streaming por rangos, sin que la complejidad del marco se interpusiera en el diseño del dominio. La elección de Angular 21 con Standalone Components y Signals ha permitido, además, escribir un cliente que se siente moderno desde el primer momento: menos ceremonia, más expresividad y un rendimiento sostenido en operación.

Las piezas que mejor ilustran el valor técnico del proyecto son probablemente las del subsistema de streaming y analíticas. Resolver de forma elegante la tensión entre las exigencias de un reproductor HTML5 nativo —peticiones *Range*, sin cabeceras personalizables— y un modelo de seguridad *stateless* basado en JWT, mediante un Service Worker que reescribe URLs y un filtro *backend* que acepta el token por dos vías sólo en la ruta de streaming, es una solución concreta a un problema real, no académico. La capa analítica dual —contador agregado en *video_vistas* y telemetría por sesión en logs— responde a la misma lógica: separar lo que se consulta mucho de lo que se acumula con detalle, para que cada tabla rinda en su terreno.

Más allá del resultado funcional, el proceso de construcción de Lockstrm ha sido un ejercicio completo de ingeniería del software: análisis de un dominio con relaciones complejas, diseño de un modelo relacional normalizado con migraciones versionadas, implementación por capas con separación clara de responsabilidades, decisiones razonadas sobre seguridad y rendimiento, contenedorización de toda la infraestructura, automatización del despliegue mediante GitHub Actions y diseño de una capa analítica útil. A nivel personal, el proyecto ha consolidado el dominio sobre tecnologías que ya formaban parte de mi caja de herramientas y ha incorporado otras —Signals, *range requests*, Service Workers como proxy de autenticación, UPSERT idempotente como patrón analítico— que abren nuevas posibilidades de cara al ejercicio profesional.

Lockstrm queda, al cierre de este TFG, en un punto de equilibrio interesante: es un producto funcional, evaluable y desplegable —con despliegue continuo en marcha— y al mismo tiempo es una base preparada para evolucionar hacia escenarios más exigentes —caché distribuida, escalado horizontal, transcoding adaptativo, suite de tests sólida— sin necesidad de reescribir su arquitectura. Esa combinación de completitud y apertura es, quizá, la mejor síntesis de lo que se buscaba al iniciar el proyecto.